

CSA15 Cloud Computing and Big Data Analytics

CO3 AT2 - Capstone Cloud Access and Service Integration Case Study Student Template

Course Code	CSA15	Course Title	Cloud Computing and Big Data Analytics
Assessment	CO3 AT2 - Capstone Cloud Access and Service Integration Case Study	Submission Type	Individual digital case-study document
Faculty	Dr C. Rajesh Babu	Employee ID	SSETSCS415
Mode	Case study / capstone design reasoning	Submission	GitHub repository link through Google Classroom

How This Template Works

Read each case, connect it to the technical ideas learned in CO3 AT1, and then extend the design to your own capstone product. The answer must show cloud access, APIs, storage, serverless/event flow and edge/fog/cloud reasoning. Do not upload passwords, private keys, API tokens or sensitive personal data.

Student and Capstone Details

Student Name	T.PRAKEERTHI
Register Number	192524279
Class / Slot	Slot B
Capstone Title	Medi Queue AI: Hospital of Queue and No-show. prediction App.
CO3 AT1 Evidence Link	
GitHub Repository Link	https://github.com/tarigondaprakeerthi/CSA1512-cloud-computing-Assessment.git

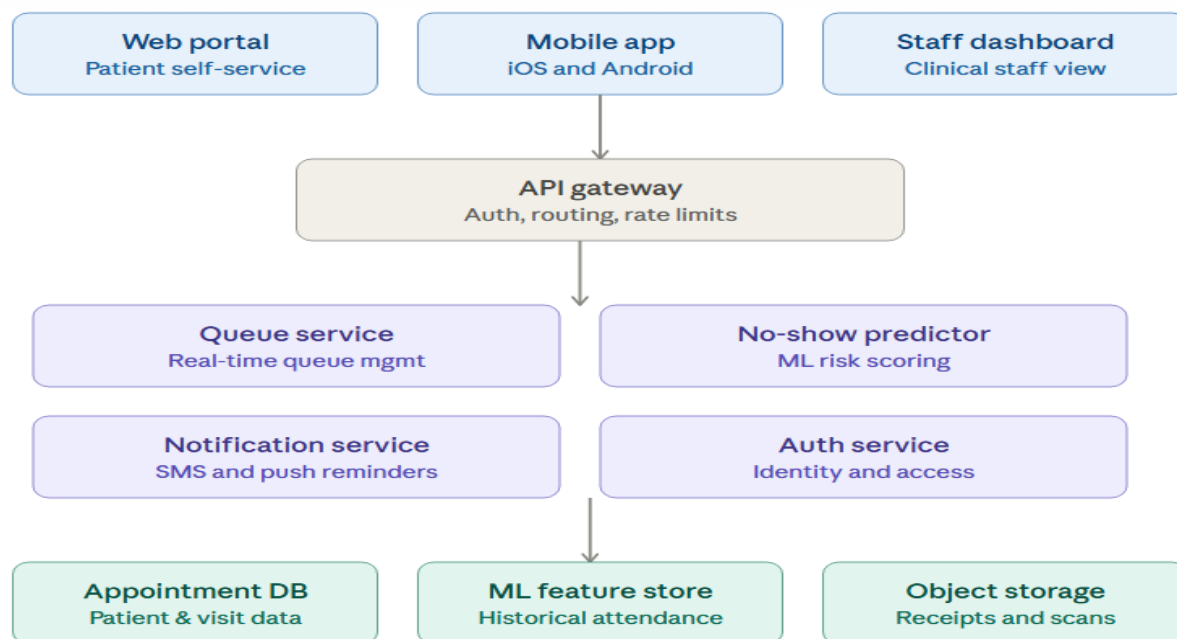
Submission Date

Case 1: Cloud-Native Backend and Web API Architecture

Scenario

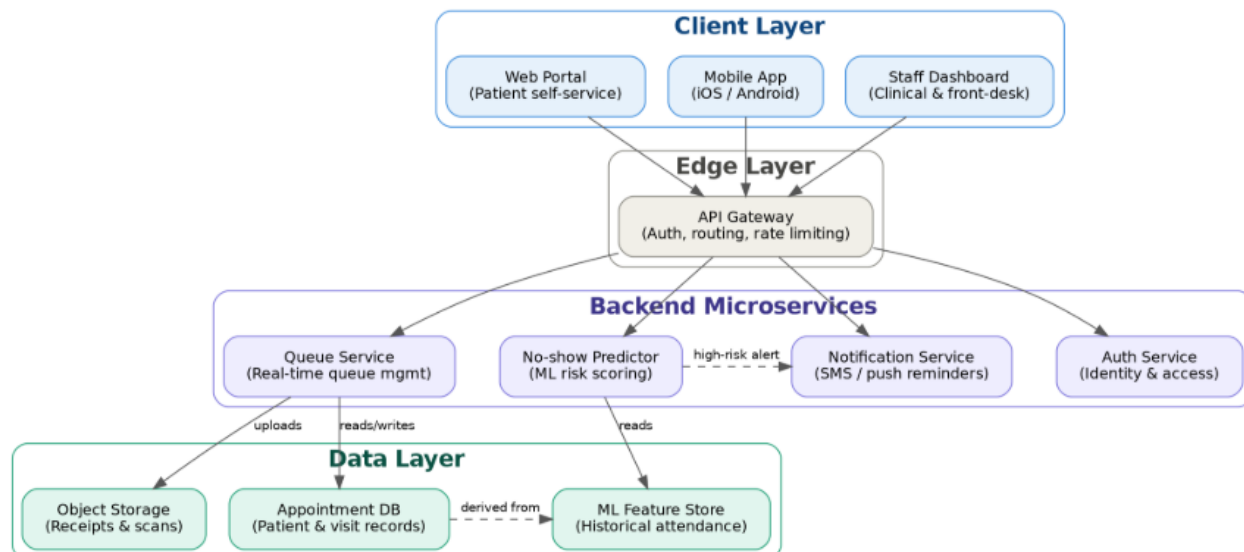
A food delivery company wants to deploy a cloud-native backend with web APIs, browser/mobile clients and shared cloud storage for receipts and delivery images. Design the architecture and explain the role of each component.

A. Case Architecture Diagram

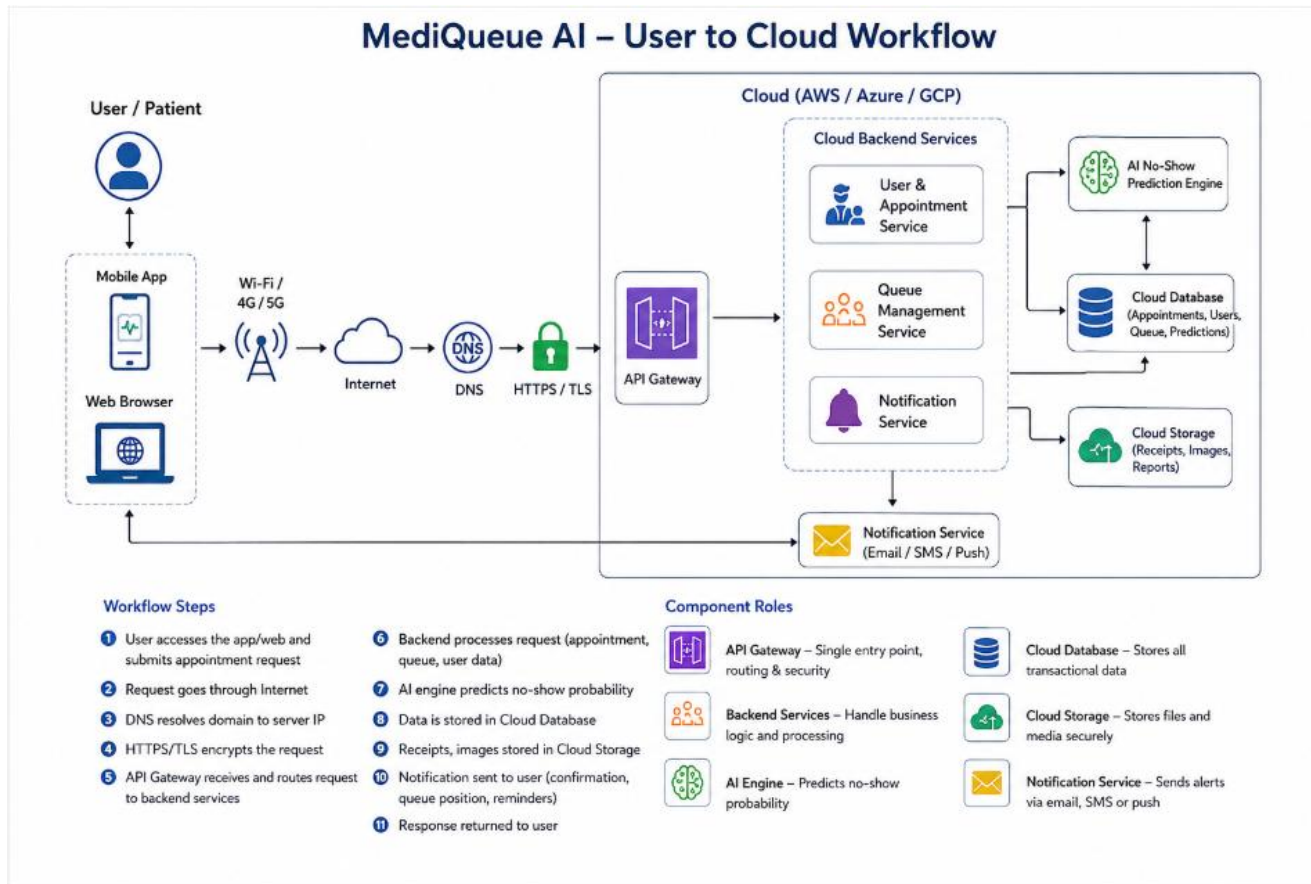


B. Component Role Explanation

MediQueue AI – Hospital Queue & No-show Prediction: Architecture



C. Capstone Extension: user-to-cloud workflow for your product



Case 2: Storage and Serverless Upload Workflow

Scenario

A media startup must decide between SAN-backed storage, object storage and serverless processing for upload workflows. Recommend a deployment model and justify it.

Object Storage +

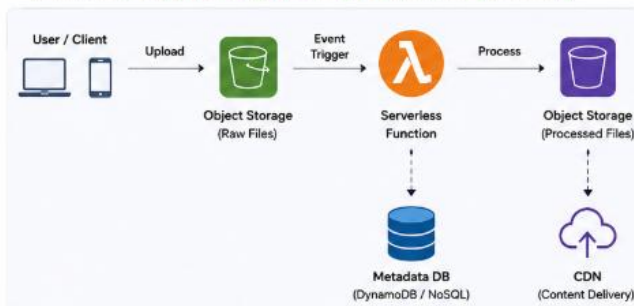
Serverless Processing is the best choice for a media startup because media files such as videos, images, and audio are large, need high scalability, and may be uploaded frequently. Serverless functions can automatically process files when they are uploaded, without managing servers.

Data / Event	Best Storage	Serverless Action	Reason
Image upload	Object Storage	Resize/compress image	Scalable and cost-effective
Video upload	Object Storage	Generate thumbnails/transcode video	Handles large media files efficiently
Audio upload	Object Storage	Convert audio format	Automatic processing after upload
User metadata	Object Storage	Validate and update metadata	Fast and flexible access

A. Storage and Serverless Recommendation

Scenario: A media startup must decide between SAN-backed storage, object storage and serverless processing for upload workflows.

Recommended Model: Object Storage + Serverless Processing



Why This Model?

- Highly Scalable**: Handles large volumes of media uploads easily.
- Cost Effective**: Pay only for storage and compute used.
- Fully Managed**: No server management or maintenance.
- Event Driven**: Automatic processing on upload.
- Durable & Secure**: Built-in redundancy and data protection.

Why Not SAN for This Use Case?

- | | | |
|--|----|---|
| SAN-Backed Storage <ul style="list-style-type: none"> ✗ High cost and complex management ✗ Not ideal for unstructured media files ✗ Less scalable for bursty workloads ✗ Requires manual provisioning | VS | Object Storage <ul style="list-style-type: none"> ✓ Built for unstructured data ✓ Massively scalable ✓ Cost efficient ✓ Works seamlessly with serverless |
|--|----|---|

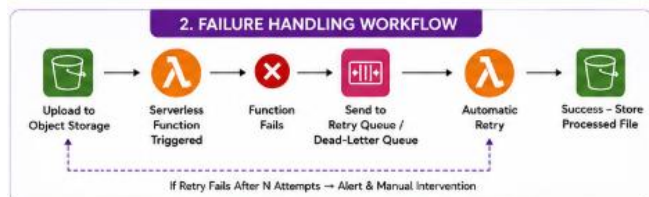
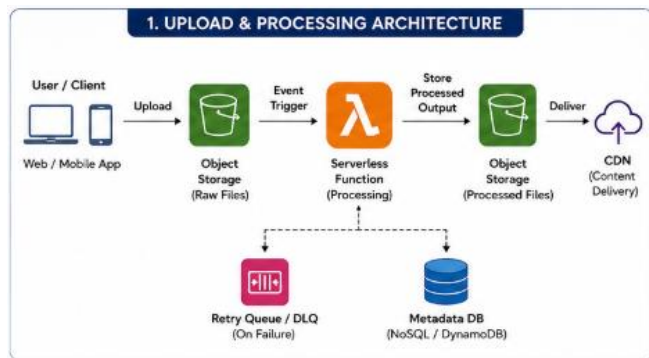
Data / Event and Serverless Workflow

Data / Event	Best Storage	Serverless Action	Reason
Image Upload	Object Storage	Resize and Compress Images	Reduces size, saves storage and bandwidth
Video Upload	Object Storage	Generate Thumbnails, Transcode Video	Supports multiple formats and smooth streaming
Audio Upload	Object Storage	Convert Audio Format, Generate Waveform	Ensures compatibility and improves playback experience
Metadata Update	NoSQL Database	Validate and Update Metadata	Fast queries and flexible schema
File Upload Event	Object Storage (Event Trigger)	Trigger Serverless Function	Event-driven processing without manual intervention
Processed Media	Object Storage (Processed Folder)	Move/Store Processed Files	Organized storage and easy retrieval
Analytics Data	NoSQL Database	Store Processing Logs and Analytics	Tracks usage, errors and performance

Conclusion

Object Storage combined with Serverless Processing is the ideal choice for media upload workflows. It provides scalability, automation, cost savings and high availability, enabling the media startup to focus on growth and user experience.

B. Failure Handling and Cost/Performance Explanation



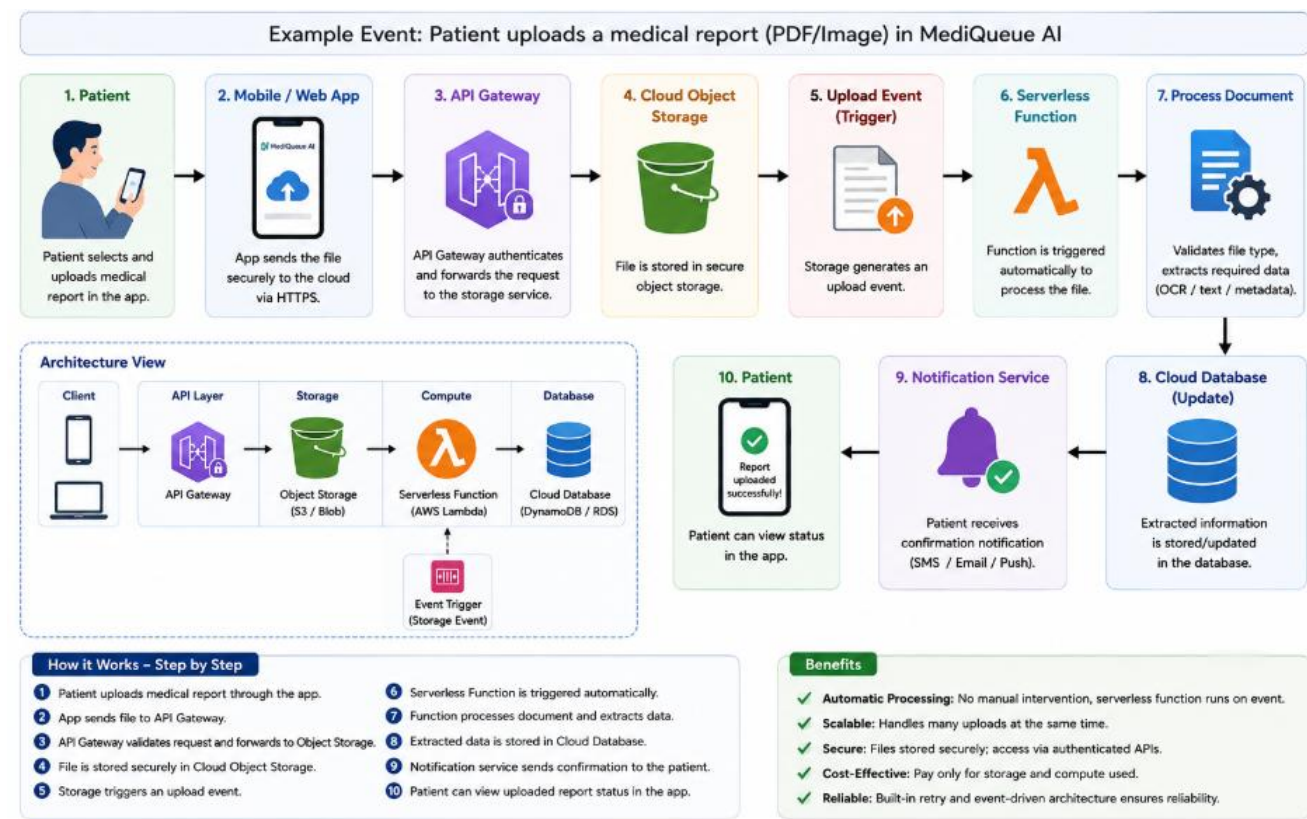
DATA / EVENT, STORAGE, ACTIONS AND REASONS			
Data / Event	Best Storage	Serverless Action	Reason
Image Upload	Object Storage	Resizes and Compress Images	Reduces size, saves storage and bandwidth
Video Upload	Object Storage	Generate Thumbnails, Transcode Video	Supports multiple formats and smooth streaming
Audio Upload	Object Storage	Convert Audio Format, Generate Waveform	Ensures compatibility and improves playback experience
Metadata Update	NoSQL Database	Validate and Update Metadata	Fast queries and flexible schema
File Upload Event	Object Storage (Event Trigger)	Trigger Serverless Function	Event-driven processing without manual intervention
Processed Media	Object Storage (Processed Folder)	Move/Store Processed Files	Organized storage and easy retrieval
Analytics Data	NoSQL Database	Store Processing Logs and Analytics	Tracks usage, errors and performance

4. SUMMARY / CONCLUSION

- ✓ Object Storage + Serverless Processing is the ideal model for media upload workflows.
- ✓ Provides automatic scalability, high availability and reliability.
- ✓ Built-in failure handling with retry mechanisms ensures no data loss.
- ✓ Cost-effective and pay-as-you-go model reduces operational cost.
- ✓ Better performance with CDN for global content delivery.

★ This architecture ensures reliable, scalable, and cost-efficient media processing for a growing media startup.

C. Capstone Extension: one upload/event workflow

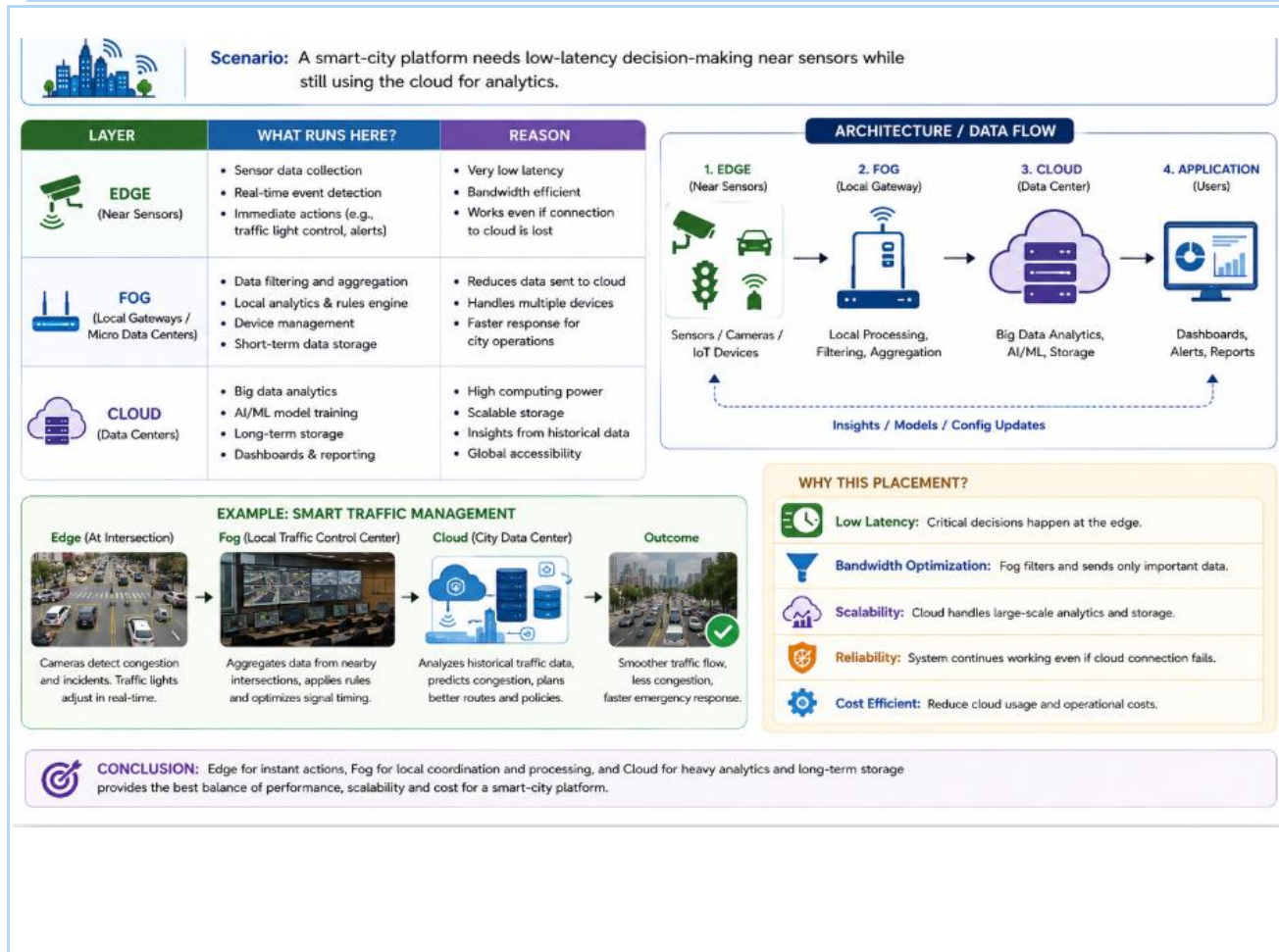


Case 3: Edge, Fog and Cloud Placement

A. Edge/Fog/Cloud Architecture Explanation

Scenario

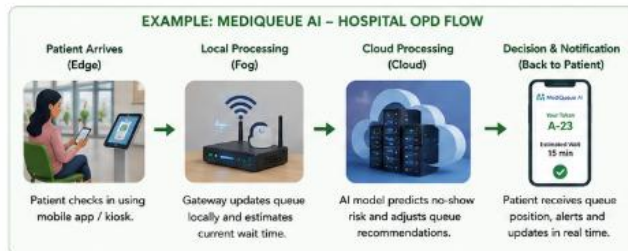
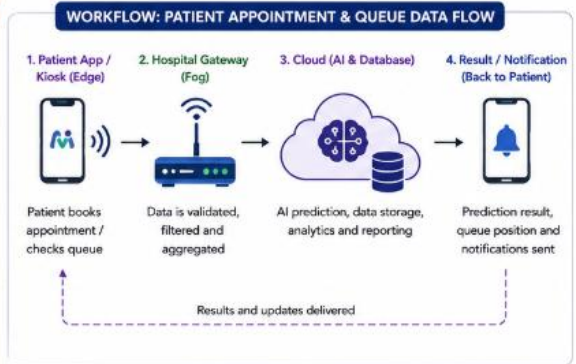
A smart-city platform needs low-latency decision-making near sensors while still using the cloud for analytics. Decide what runs at edge, fog and cloud..



B. Capstone Extension or Direct Cloud Access Justification

Capstone Extension: Edge, Fog and Cloud Placement for MediQueue AI (Hospital OP Queue & No-Show Prediction)

LAYER	MEDIQUEUE AI COMPONENT	WHAT RUNS HERE?	PURPOSE / BENEFITS
EDGE (Near Users / Devices)	- Patient Mobile App - Hospital Kiosks - IoT Sensors (Waiting Area Count, Token Scanners, etc.)	- Data collection - Local validation - Quick response - Temporary caching	- Very low latency - Instant updates for patients - Works even with intermittent internet
FOG (Hospital / Local Gateway)	- Hospital Local Server / Gateway - Queue Management Service - Data Filtering & Aggregation - Rule Engine (Local Alerts)	- Aggregate data from multiple devices - Filter unnecessary data - Local processing & decision - Store short-term data	- Reduces bandwidth usage - Faster response inside hospital - Continues working if cloud is slow
CLOUD (Cloud Data Center)	- AI No-Show Prediction Engine - Central Database - Analytics & Reporting - Admin Dashboard - Notification Service	- Big data processing - AI/ML model training and inference - Long-term storage - Reports and analytics	- High scalability - Powerful AI/ML capabilities - Global access - Secure & reliable storage



JUSTIFICATION		
	Low Latency Response	Edge and Fog layers provide instant updates for patients and hospital staff, improving experience.
	Bandwidth Optimization	Fog layer filters and aggregates data, reducing unnecessary data transfer to the cloud.
	Scalability	Cloud handles heavy AI processing, storage and analytics, which can scale on demand.
	Reliability & Resilience	System continues working even if cloud connection is slow or down, thanks to edge and fog processing.
	Cost Efficiency	Only important and aggregated data is sent to the cloud, reducing storage and network costs.

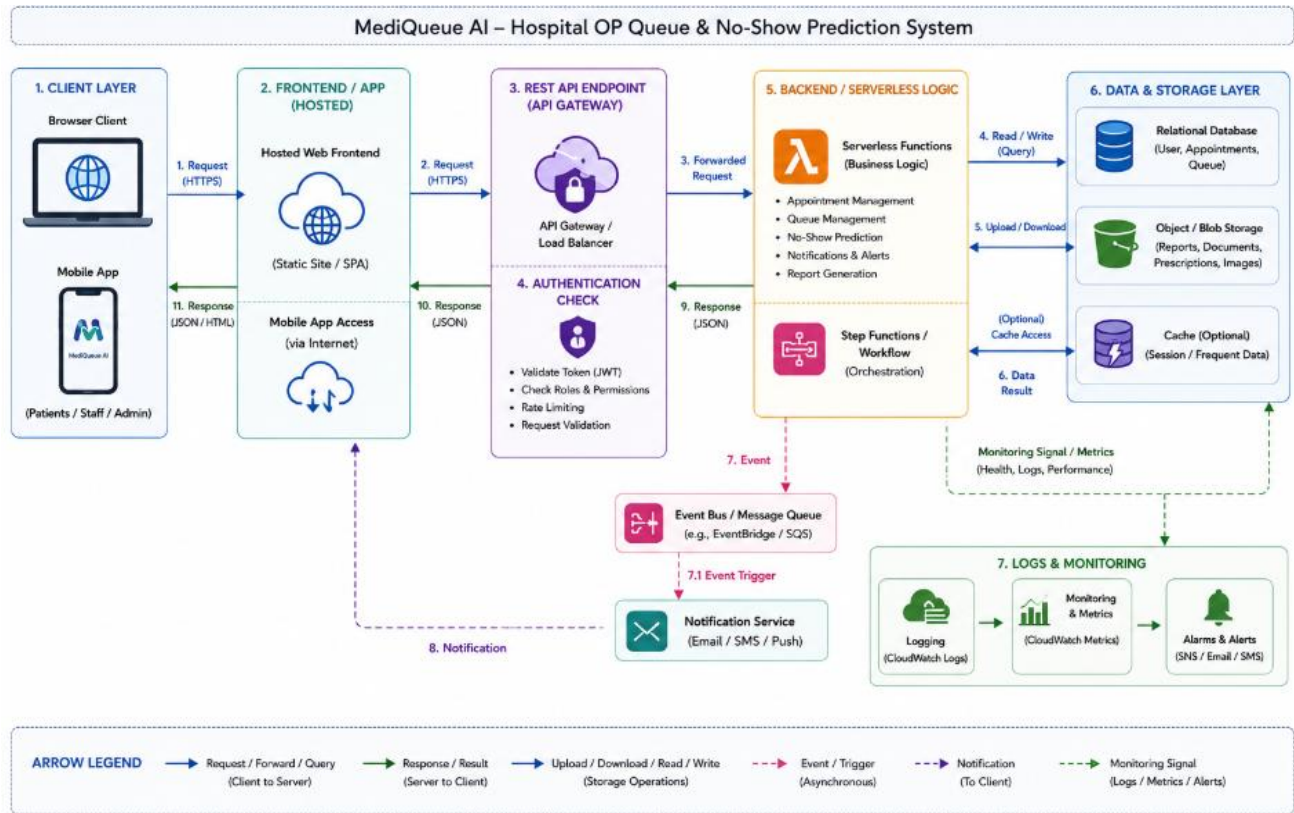


CONCLUSION: Using Edge for quick actions, Fog for local coordination and filtering, and Cloud for advanced AI and analytics gives MediQueue AI low latency, scalability, reliability and cost efficiency. This multi-layer architecture is better than direct cloud access alone for a smart hospital system.

Final Capstone Cloud Access Architecture

- ☐ Show Browser/Mobile Client.
- ☐ Show hosted frontend or app access path.
- ☐ Show REST API endpoint and authentication check.
- ☐ Show backend/serverless logic.
- ☐ Show database, object/blob storage, logs and monitoring.
- ☐ Label the arrows: request, response, upload, event, notification or monitoring signal.

Final Diagram and Explanation



Submission Checklist

- ☐ Template completed in DOCX format.
- ☐ Diagrams pasted clearly in the document.
- ☐ GitHub repository contains DOCX/PDF, diagrams and README.
- ☐ Google Classroom submission contains GitHub link.
- ☐ Sensitive credentials are not visible anywhere.

Student Assurance

Declaration

I confirm that this case-study answer, cloud access design, diagrams and capstone extensions were prepared individually by me for the stated capstone title.

Student Signature: T.PRAKEERTHI Date: 12/08/2026